Módulo 6 - Aulas 1 e 2

Tarefas

No final deste módulo você deve submeter em um ÚNICO arquivo PDF os seguintes prints:

- Atividade 6.1: passo 8.
- Atividade 6.2: passo 7.
- Atividade 6.3: passo 8.
- Atividade 6.4: passo 6.
- Atividade 6.5: passo 8.

Regras para a elaboração do documento:

1. Antes de cada Print, adicione obrigatoriamente uma frase explicativa que sinalize do que se trata o print. A inserção de prints sem a devida frase explicativa será considerada como tentativa de atrapalhar a correção do instrutor e será penalizada a critério do instrutor. Exemplo:

Print da atividade 6.1: [Imagem com o Print]

2. Os Prints devem ser tirados da TELA CHEIA. Quando tirados da VM da AWS, devem ser capturados **obrigatoriamente** da tela cheia clicando no botão "Screenshot" → "Take screenshot" da barra de ferramentas do Hypervisor da AWS.
3. Insira **somente a quantidade de Prints solicitados por atividade** usando exclusivamente 1 página por print. **A página do documento onde você vai inserir o Print deve estar com a orientação no modo PAISAGEM** ´para termos melhor aproveitamento do espaço. Ou seja, seu documento deverá ter a mesma quantidade de páginas que a quantidade do total de Prints! A inserção de prints desnecessários será considerada como tentativa de atrapalhar a correção do instrutor e será penalizada com nota 0.

4. Apresente Prints legíveis e com tamanho correto para fácil leitura. O envio de prints com letras minúsculas poderá ser considerado como tentativa de atrapalhar a correção do instrutor e será penalizada a critério do instrutor.

Atividade 6.1 – Exemplo de Reuso de Código em Python no Kali Linux

Nesta atividade, vamos usar o Kali Linux para exemplificar um caso de Reuso de Código em Python para o redimensionamento de figuras em processamento de imagens.

Vamos começar inicializando nosso Kali Linux via RDP ao IP: 192.168.98.40, com usuário “aluno” e senha “rnpesr”.

1. Abra o Terminal e execute o seguinte comando (com senha “rnpesr”) para ser super usuário:

```
┌──(aluno㉿kali)-[~]
└─$ sudo -i
[sudo] senha para aluno:
```

2. Navegue até a pasta Documentos:

```
┌──(root㉿kali)-[~]
└─# cd /home/aluno/Documentos

┌──(root㉿kali)-[/home/aluno/Documentos]
└─# ls
```

3. Baixe a imagem My_shoe.jpg via Terminal:

```
┌──(root㉿kali)-[/home/aluno/Documentos]
└─# wget https://upload.wikimedia.org/wikipedia/commons/0/01/
My_shoe.jpg
--2024-02-13 17:24:33--  https://upload.wikimedia.org/wikipedia/
commons/0/01/My_shoe.jpg
Resolvendo upload.wikimedia.org (upload.wikimedia.org)...
208.80.154.240, 2620:0:861:ed1a::2:b
Conectando-se a upload.wikimedia.org (upload.wikimedia.org)|
208.80.154.240|:443... conectado.
A requisição HTTP foi enviada, aguardando resposta... 200 OK
Tamanho: 1297035 (1,2M) [image/jpeg]
Salvando em: “My_shoe.jpg”

My_shoe.jpg
100%[===========================================>]   1,24M  --.-KB/s
em 0,1s

2024-02-13 17:24:33 (8,28 MB/s) - “My_shoe.jpg” salvo [1297035/1297035]


┌──(root㉿kali)-[/home/aluno/Documentos]
└─# ls
My_shoe.jpg
```

4. Agora, abra o Thunar (clique em “Aplicativos” → “Gerenciador de arquivos”), navegue até "/home/aluno/Documentos" e com 2 cliques visualize o arquivo My_shoe.jpg. Após a visualização, feche o Visualizador de Imagens.
5. Abra o programa Editor de Texto Mousepad (clique em “Aplicativos” → “Aplicações habituais” → “Acessórios” → “Editor de Texto”) e cole o seguinte código em Python:

```
from PIL import Image

# Abre uma imagem existente
imagem_original = Image.open("My_shoe.jpg")

# Redimensiona a imagem para 300x300 pixels
imagem_redimensionada = imagem_original.resize((300, 300))

# Salva a imagem redimensionada
imagem_redimensionada.save("My_shoe_redimensionado.jpg")

# Fecha a imagem original
imagem_original.close()
```

O código mostrado acima faz o **reuso de código utilizando componentes de software já testados e validados**, em vez de criar funcionalidades do zero. Veja os detalhes do código mostrado acima:

- **from PIL import Image**: Importa a classe Image da biblioteca Pillow, que é usada para trabalhar com imagens.
- **imagem_original = Image.open("My_shoe.jpg")**: Abre uma imagem chamada "My_shoe.jpg" no diretório atual. A imagem original é carregada e atribuída à variável imagem_original.
- **imagem_redimensionada = imagem_original.resize((300, 300))**: O método resize() é chamado na imagem original (imagem_original) para redimensioná-la para uma nova dimensão de 300×300 pixels. O resultado do redimensionamento é atribuído à variável imagem_redimensionada.
- **imagem_redimensionada.save("My_shoe_redimensionado.jpg")**: Salva a imagem redimensionada no arquivo "My_shoe_redimensionado.jpg" no diretório atual. A imagem redimensionada é escrita no disco com o novo nome.
- **imagem_original.close()**: Fecha o arquivo da imagem original após ter sido carregado e redimensionado. Isso é uma boa prática para liberar recursos após a manipulação de imagens.

6. Salve o código (clique em “Arquivo” → “Salvar Como”) na pasta Documentos (/home/aluno/Documentos) com o nome *redimensionar_imagem.py*. Não se esqueça de mudar a opção “Arquivos de Texto” para “Todos os arquivos”, acima do botão “Salvar”.
7. Feche o Editor de Texto Mousepad e volte ao Terminal. Execute o código criado no passo 5:

```
┌──(root㉿kali)-[/home/aluno/Documentos]
└─# python redimensionar_imagem.py
```

8. Neste momento, uma nova imagem do tamanho de 300×300 pixels deve ter sido criada. Confira (**NÃO SE ESQUEÇA DE PRINTAR ESTE PASSO!**):

```
┌──(root㉿kali)-[/home/aluno/Documentos]
└─# ls
My_shoe.jpg  My_shoe_redimensionado.jpg  redimensionar_imagem.py
```

9. Volte à janela do Thunar e visualize o arquivo My_shoe_redimensionado.jpg. Veja que esse arquivo é muito menor que o arquivo original My_show.jpg.
10. Apague os arquivos usados na pasta Documentos, feche o Thunar e o Terminal:

```
┌──(root㉿kali)-[/home/aluno/Documentos]
└─# rm *
zsh: sure you want to delete all 3 files in /home/aluno/Documentos
[yn]? y


┌──(root㉿kali)-[/home/aluno/Documentos]
└─# ls
```

Parabéns! Agora você entendeu o conceito de reutilização de código!

Atividade 6.2 – Conhecendo o SDK do Java (JDK) e um IDE no Kali Linux

Nesta atividade, vamos usar o Kali Linux para instalar o SDK do Java, conhecido como Java Development Kit (JDK), e o IDE Netbeans. Com o NetBeans você poderá começar a desenvolver programas em Java!

Vamos começar inicializando nosso Kali Linux via RDP ao IP: 192.168.98.40, com usuário “aluno” e senha “rnpesr”.

1. Abra o Terminal e execute o seguinte comando (com senha “rnpesr”) para ser super usuário:

```
┌──(aluno㉿kali)-[~]
└─$ sudo -i
[sudo] senha para aluno:
```

2. No seu laboratório, você terá o JDK funcionando. Verifique a versão do seu JDK:

```
┌──(root㉿kali)-[~]
└─# java --version
openjdk 23-ea 2024-09-17
OpenJDK Runtime Environment (build 23-ea+20-Debian-1)
OpenJDK 64-Bit Server VM (build 23-ea+20-Debian-1, mixed mode, sharing)
```

3. Navegue até a pasta Netbeans do nosso curso:

```
┌──(root㉿kali)-[~]
└─# cd /curso/netbeans
```

4. Verifique que o arquivo está presente na pasta Netbeans:

```
┌──(root㉿kali)-[/home/curso/netbeans]
└─# ls
apache-netbeans_19-1_all.deb
```

5. Em seguida, instalaremos o arquivo baixado com o comando dpkg:

```
┌──(root㉿kali)-[/curso/netbeans]
└─# dpkg -i apache-netbeans_19-1_all.deb
Selecionando pacote previamente não selecionado apache-netbeans.
(Lendo banco de dados ... 364754 arquivos e diretórios atualmente
instalados).
Preparando para desempacotar apache-netbeans_19-1_all.deb ...
Desempacotando apache-netbeans (19-1) ...
Configurando apache-netbeans (19-1) ...
Processando gatilhos para kali-menu (2025.3.2) ...
Processando gatilhos para desktop-file-utils (0.28-1) ...
Processando gatilhos para hicolor-icon-theme (0.18-2) ...
```

6. Na barra de tarefas superior, abra o Apache NetBeans (clique em “Aplicativos” → “Aplicativos habituais” → “Desenvolvimento” → “Apache NetBeans”).
7. Finalmente, o NetBeans IDE será aberto! Explore o NetBeans (**NÃO SE ESQUEÇA DE PRINTAR ESTE PASSO!**) .
8. Feche o Apache NetBeans e o Terminal.

Parabéns! Agora você poderá criar aplicações em Java com o JDK por meio do NetBeans!

Atividade 6.3 – Automação de Tarefas com Python no Kali Linux

Nesta atividade, vamos usar o Python para automatizar tarefas no Kali Linux.

Vamos começar inicializando nosso Kali Linux via RDP ao IP: 192.168.98.40, com usuário “aluno” e senha “rnpesr”.

1. Abra o Terminal e execute o seguinte comando (com senha “rnpesr”) para ser super usuário:

```
┌──(aluno㉿kali)-[~]
└─$ sudo -i
[sudo] senha para aluno:
```

2. Abra o programa Editor de Texto Mousepad (clique em “Aplicativos” → “Aplicações habituais” → “Acessórios” → “Editor de Texto”) e cole o seguinte código em Python:

```
import os
import socket

# Função para coletar informações do sistema
def coletar_informacoes_sistema():
    hostname = socket.gethostname()
    kernel_version = os.uname().release
    cpu_info = os.popen('cat /proc/cpuinfo | grep "model name" |
uniq').read().strip()
    mem_info = os.popen('free -h | grep "Mem.:"').read().strip()
    disk_space = os.popen('df -h').read()

    informacoes = {
        "Hostname": hostname,
        "Kernel Version": kernel_version,
        "CPU Info": cpu_info,
        "Memory Info": mem_info,
        "Disk Space": disk_space
    }

    return informacoes

# Função para gerar um relatório
def gerar_relatorio(informacoes):
    relatorio = "Relatório do Sistema Linux\n\n"
    for chave, valor in informacoes.items():
        relatorio += f"{chave}: {valor}\n"

    with open("relatorio_linux.txt", "w") as arquivo:
        arquivo.write(relatorio)

if __name__ == "__main__":
    informacoes_sistema = coletar_informacoes_sistema()
    gerar_relatorio(informacoes_sistema)
    print("Relatório gerado com sucesso em 'relatorio_linux.txt'")
```

Veja os detalhes desse código Python:

- **import os**: Importa o módulo os, que fornece funções para interagir com o sistema operacional, como acessar comandos do sistema.
- **import socket**: Importa o módulo socket, que fornece funções para lidar com operações de rede e informações sobre o host.

A seguir, definimos uma função chamada coletar_informacoes_sistema(). Esta função é responsável por coletar informações do sistema Linux. As informações coletadas incluem:

- hostname: O nome do host, obtido usando a função socket.gethostname() do módulo socket.
- kernel_version: A versão do kernel, obtida usando os.uname().release. O uname() é uma chamada de sistema que fornece informações sobre o sistema operacional.
- cpu_info: Informações da CPU obtidas através do comando cat /proc/cpuinfo | grep "model name" | uniq. Isso extrai as informações do arquivo /proc/cpuinfo relacionadas ao modelo da CPU.
- mem_info: Informações de uso de memória obtidas usando o comando free -h | grep "Mem:". Isso exibe as informações de memória do sistema.
- disk_space: Espaço em disco obtido com o comando df -h, que fornece informações sobre o espaço em disco.

As informações coletadas são armazenadas em um dicionário chamado informações, onde as chaves são nomes das informações e os valores são os dados coletados.

- A função coletar_informacoes_sistema() retorna o dicionário informacoes como resultado.
- Em seguida, definimos uma função chamada gerar_relatorio(informacoes). Esta função aceita o dicionário de informações como argumento e gera um relatório formatado.
- O relatório começa com a string "Relatório do Sistema Linux\n\n".
- Usando um loop for, o código percorre o dicionário de informações e formata as chaves e valores em linhas de texto, que são adicionadas ao relatório.
- O relatório é então escrito em um arquivo chamado "relatorio_linux.txt" usando um bloco with open(...). O modo "w" indica que o arquivo será aberto para escrita, e o bloco with garante que o arquivo seja fechado automaticamente após a escrita.
- Por fim, a função principal é protegida pela condição if **name** == "**main**". Isso garante que o código dentro do bloco só seja executado se o script for executado diretamente (não quando importado como módulo em outro script).

- Dentro do bloco principal, as informações do sistema são coletadas chamando coletar_informacoes_sistema(), o relatório é gerado chamando gerar_relatorio(informacoes_sistema), e uma mensagem é exibida no console informando que o relatório foi gerado com sucesso.

3. Salve o código (clique em “Arquivo” → “Salvar Como”) na pasta Documentos (/home/aluno/Documentos) com o nome “dados_linux.py”. Não se esqueça de mudar a opção “Arquivos de Texto” para “Todos os arquivos”, acima do botão “Salvar”.
4. Feche o Mousepad. No Terminal, navegue até a pasta Documentos:

```
┌──(root㉿kali)-[~]
└─# cd /home/aluno/Documentos/
```

5. Execute o arquivo Python criado no passo 2:

```
┌──(root㉿kali)-[/home/aluno/Documentos]
└─# python dados_linux.py
Relatório gerado com sucesso em 'relatorio_linux.txt'
```

6. Veja que o relatório foi gerado em txt:

```
┌──(root㉿kali)-[/home/aluno/Documentos]
└─# ls
dados_linux.py  relatorio_linux.txt
```

7. Visualize o relatório com o comando:

```
┌──(root㉿kali)-[/home/aluno/Documentos]
└─# cat relatorio_linux.txt
```

8. Veja os principais detalhes de Hardware do seu computador no relatório (**NÃO SE ESQUEÇA DE PRINTAR ESTE PASSO!**):

```
┌──(root㉿kali)-[/home/aluno/Documentos]
└─# cat relatorio_linux.txt
Relatório do Sistema Linux

Hostname: kali
Kernel Version: 6.12.38+kali-cloud-amd64
CPU Info: model name    : AMD EPYC 7571
Memory Info: Mem.:          3,8Gi       881Mi       924Mi        15Mi
2,3Gi       2,9Gi
Disk Space: Sist. Arq.       Tam. Usado Disp. Uso% Montado em
udev             1,9G     0  1,9G   0% /dev
tmpfs            388M  956K  387M   1% /run
/dev/nvme0n1p1    20G   19G  101M 100% /
tmpfs            1,9G  4,0K  1,9G   1% /dev/shm
tmpfs            5,0M     0  5,0M   0% /run/lock
tmpfs            1,9G   56K  1,9G   1% /tmp
tmpfs            1,0M     0  1,0M   0% /run/credentials/systemd-
journald.service
/dev/nvme0n1p15  124M  294K  124M   1% /boot/efi
tmpfs            1,0M     0  1,0M   0% /run/credentials/serial-
getty@ttyS0.service
tmpfs            1,0M     0  1,0M   0% /run/credentials/
getty@tty1.service
tmpfs            388M  120K  388M   1% /run/user/0
tmpfs            388M  140K  388M   1% /run/user/1001
```

A saída do comando apresenta:

- **Hostname**: O nome do host é "kali", indicando o nome do computador.
- **Kernel Version**: A versão do kernel é 6.12.38+kali-cloud-amd64, que representa a versão atual do núcleo (kernel) do sistema operacional. Essa versão é otimizada para ambientes em nuvem e arquitetura AMD64, sendo uma compilação específica do Kali Linux.
- **CPU Info**: Esta seção fornece detalhes sobre o processador utilizado no sistema. No caso, trata-se de um AMD EPYC 7571, um processador de alto desempenho voltado para servidores e ambientes virtuais. Isso sugere que o sistema está rodando em uma infraestrutura de nuvem ou servidor virtual com hardware robusto.
- **Memory Info**: Esta seção apresenta informações sobre a memória disponível no sistema.

- **Disk Space**: Mostra o estado dos sistemas de arquivos montados no sistema. As colunas significam: Sist. Arq.: Nome do dispositivo ou tipo de sistema de arquivos. Tam.: Tamanho total do espaço. Usado: Espaço ocupado. Disp.: Espaço disponível. Uso%: Percentual de uso. Montado em: Diretório onde o sistema de arquivos está montado.
- **Valores de disco**:
- udev: Sistema de arquivos virtual do kernel para gerenciamento dinâmico de dispositivos. Usa 0% do seu espaço alocado de 1,9G.
- tmpfs (/run): Sistema de arquivos temporário armazenado na memória RAM. Usa apenas 956K de 388M, destinado a dados de execução do sistema.
- /dev/nvme0n1p1: Partição principal do disco (provavelmente SSD NVMe) com capacidade total de 20G. Está com 19G usados e apenas 101M livres, resultando em 100% de uso. Esse é um alerta crítico, pois o sistema pode parar de funcionar corretamente se não houver espaço para logs, cache ou atualizações.
- tmpfs (/dev/shm): Área de memória compartilhada (shared memory), com 1,9G de tamanho e uso desprezível (4,0K).
- tmpfs (/run/lock): Usado para travas de recursos entre processos. Com 5,0M, está completamente livre.
- tmpfs (/tmp): Diretório temporário em RAM, útil para segurança e desempenho. Usa apenas 56K de 1,9G.
- tmpfs (credentials): Sistemas de arquivos virtuais pequenos (1,0M) usados pelo systemd para isolar credenciais de serviços específicos. Todos estão vazios.
- /dev/nvme0n1p15: Partição EFI de inicialização, com 124M. Apenas 294K são usados, o que é normal, pois contém apenas arquivos essenciais de boot.
- tmpfs (/run/user/0): Espaço temporário para o usuário root, com 388M de capacidade e uso mínimo (120K).
- tmpfs (/run/user/1001): Espaço temporário para o usuário com ID 1001 (geralmente o primeiro usuário não-root). Também usa apenas 140K do espaço disponível.

9. Apague os arquivos da pasta Documentos e feche e o Terminal:

```
┌──(root㉿kali)-[/home/aluno/Documentos]
└─# ls
dados_linux.py  relatorio_linux.txt

┌──(root㉿kali)-[/home/aluno/Documentos]
└─# rm *
zsh: sure you want to delete all 2 files in /home/aluno/Documentos
[yn]? y

┌──(root㉿kali)-[/home/aluno/Documentos]
└─# ls
```

Parabéns! Agora você sabe como automatizar relatórios de hardware de computadores em Linux!

Atividade 6.4 – Conhecendo o Antivírus ClamAV e sua Assinatura de Malware no Kali Linux

Nesta atividade, vamos explorar o Antivírus ClamAV no Kali Linux para rodar um escaneamento de uma pasta e conhecer a sua base de assinatura de malware.

Vamos começar inicializando nosso Kali Linux via RDP ao IP: 192.168.98.40, com usuário “aluno” e senha “rnpesr”.

1. Abra o Terminal e execute o seguinte comando (com senha “rnpesr”) para ser super usuário:

```
┌──(aluno㉿kali)-[~]
└─$ sudo -i
[sudo] senha para aluno:
```

2. Veja as opções que podem ser executadas com o comando:

```
┌──(root㉿kali)-[~]
└─# clambc -h

                        Clam AntiVirus: Bytecode Testing Tool 1.4.3
           By The ClamAV Team: https://www.clamav.net/
about.html#credits
           (C) 2024 Cisco Systems, Inc.

    clambc <file> [function] [param1 ...]

    --help                 -h         Show this help
    --version              -V         Show version
    --debug                           Show debug
    --force-interpreter    -f         Force using the interpreter
instead of the JIT
    --trust-bytecode       -t         Trust loaded bytecode (default
yes)
    --info                 -i         Print information about bytecode
    --printsrc             -p         Print bytecode source
    --printbcir            -c         Print IR of bytecode signature
    --input                -c         Input file to run the bytecode on
    --trace <level>        -T         Set bytecode trace level 0..7
(default 7)
    --no-trace-showsource  -s         Don't show source line during
tracing
    --statistics=bytecode             Collect and print bytecode
execution statistics
    file                              File to test

**Caution**: You should NEVER run bytecode signatures from untrusted
sources.
Doing so may result in arbitrary code execution.
```

3. Agora, vamos explorar o utilitário *sigtool* do ClamAV, que é uma ferramenta de linha de comando para trabalhar com assinaturas de vírus e realizar várias operações relacionadas à gestão de assinaturas:

```
┌──(root㉿kali)-[~]
└─# sigtool -h

                      Clam AntiVirus: Signature Tool 1.4.3
           By The ClamAV Team: https://www.clamav.net/
about.html#credits
           (C) 2024 Cisco Systems, Inc.

    sigtool [options]

    --help                  -h              Show this help
    --version               -V              Print version number and
exit
    --quiet                                 Be quiet, output only error
messages
    --debug                                 Enable debug messages
    --stdout                                Write to stdout instead of
stderr. Does not affect 'debug' messages.
    --hex-dump                              Convert data from stdin to a
hex
                                            string and print it on
stdout
    --md5 [FILES]                           Generate MD5 checksum from
stdin
                                            or MD5 sigs for FILES
    --sha1 [FILES]                          Generate SHA1 checksum from
stdin
                                            or SHA1 sigs for FILES
    --sha256 [FILES]                        Generate SHA256 checksum
from stdin
                                            or SHA256 sigs for FILES
    --mdb [FILES]                           Generate .mdb (section hash)
sigs
    --imp [FILES]                           Generate .imp (import table
hash) sigs
    --fuzzy-img FILE(S)                     Generate image fuzzy hash
for each file
    --html-normalise=FILE                   Create normalised parts of
HTML file
    --ascii-normalise=FILE                  Create normalised text file
from ascii source
    --utf16-decode=FILE                     Decode UTF16 encoded files
    --info=FILE             -i FILE         Print database information
    --build=NAME [cvd] -b NAME              Build a CVD file
    --max-bad-sigs=NUMBER                   Maximum number of mismatched
signatures
                                            When building a CVD.
Default: 3000
```

```
Default: 3000
    --flevel=FLEVEL                         Specify a custom flevel.
                                            Default: 213
    --cvd-version=NUMBER                    Specify the version number
to use for
                                            the build. Default is to use
the value+1
                                            from the current CVD in --
datadir.
                                            If no datafile is found the
default
                                            behaviour is to prompt for a
version
                                            number, this switch will
prevent the
                                            prompt.  NOTE: If a CVD is
found in the
                                            --datadir its version+1 is
used and
                                            this value is ignored.
    --no-cdiff                              Don't generate .cdiff file
    --unsigned                              Create unsigned database
file (.cud)
    --hybrid                                Create a hybrid (standard
and bytecode) database file
    --print-certs=FILE                      Print Authenticode details
from a PE
    --server=ADDR                           ClamAV Signing Service
address
    --datadir=DIR                           Use DIR as default database
directory
    --unpack=FILE           -u FILE         Unpack a CVD/CLD file
    --unpack-current=SHORTNAME              Unpack local CVD/CLD into
cwd
    --list-sigs[=FILE]      -l[FILE]        List signature names
    --find-sigs=REGEX       -fREGEX         Find signatures matching
REGEX
    --decode-sigs                           Decode signatures from stdin
    --test-sigs=DATABASE TARGET_FILE        Test signatures from
DATABASE against
                                            TARGET_FILE
    --vba=FILE                              Extract VBA/Word6 macro code
    --vba-hex=FILE                          Extract Word6 macro code
with hex values
    --diff=OLD NEW          -d OLD NEW      Create diff for OLD and NEW
CVDs
    --compare=OLD NEW       -c OLD NEW      Show diff between OLD and
NEW files in
                                            cdiff format
```

```
                                                    cdiff format
    --run-cdiff=FILE       -r FILE          Execute update script FILE
in cwd
    --verify-cdiff=DIFF CVD/CLD             Verify DIFF against CVD/CLD
    --tempdir=DIRECTORY                     Create temporary files in
DIRECTORY
    --leave-temps[=yes/no(*)]               Do not remove temporary
files
```

O comando mostrado acima contém os seguintes parâmetros:

- **--help ou -h**: Exibe a ajuda, mostrando uma breve descrição das opções disponíveis e informações gerais sobre o sigtool.
- **--version ou -V**: Exibe o número da versão do sigtool e encerra a execução.
- **--quiet**: Suprime a saída padrão, exibindo apenas mensagens de erro.
- **--debug**: Ativa a saída de mensagens de depuração.
- **--stdout**: Direciona a saída para a saída padrão (stdout) em vez da saída de erro padrão (stderr). Isso não afeta as mensagens de depuração.
- **--hex-dump**: Converte dados da entrada padrão (stdin) em uma representação hexadecimal e imprime na saída padrão.
- **--md5 [FILES]**: Gera somas de verificação MD5 de arquivos ou da entrada padrão. Pode aceitar uma lista de arquivos como argumento.
- **--sha1 [FILES]**: Gera somas de verificação SHA1 de arquivos ou da entrada padrão.
- **--sha256 [FILES]**: Gera somas de verificação SHA256 de arquivos ou da entrada padrão.
- **--mdb [FILES]**: Gera assinaturas .mdb (hash de seção) para arquivos ou da entrada padrão.
- **--imp [FILES]**: Gera assinaturas .imp (hash da tabela de importação) para arquivos ou da entrada padrão.
- **--fuzzy-img FILE(S)**: Gera um hash de imagem fuzzy para cada arquivo. Pode aceitar uma lista de arquivos como argumento.
- **--html-normalise=FILE**: Cria partes normalizadas de arquivos HTML.
- **--ascii-normalise=FILE**: Cria um arquivo de texto normalizado a partir de uma fonte ASCII.
- **--utf16-decode=FILE**: Decodifica arquivos codificados em UTF16.
- **--info=FILE ou -i FILE**: Exibe informações sobre um banco de dados.

- **--build=NAME [cvd] ou -b NAME: Constrói um arquivo de banco de dados CVD.
- **--max-bad-sigs=NUMBER**: Define o número máximo de assinaturas discordantes ao construir um CVD.
- **--flevel=FLEVEL**: Especifica um nível de flevel personalizado.
- **--cvd-version=NUMBER**: Especifica a versão do banco de dados ao construir um CVD.
- **--no-cdiff**: Não gera um arquivo .cdiff.
- **--unsigned**: Cria um arquivo de banco de dados não assinado (.cud).
- **--hybrid**: Cria um arquivo de banco de dados híbrido, combinando assinaturas padrão e bytecode.
- **--print-certs=FILE**: Imprime detalhes Authenticode de um arquivo PE.
- **--server=ADDR**: Especifica o endereço do serviço de assinatura ClamAV.
- **--datadir=DIR**: Especifica o diretório padrão para bancos de dados.
- **--unpack=FILE ou -u FILE**: Desempacota um arquivo CVD/CLD.
- **--unpack-current=SHORTNAME**: Desempacota o CVD/CLD local no diretório atual.
- **--list-sigs[=FILE] ou -l[FILE]**: Lista nomes de assinaturas no banco de dados.
- **--find-sigs=REGEX ou -fREGEX**: Encontra assinaturas que correspondem a um padrão de expressão regular.
- **--decode-sigs**: Decodifica assinaturas da entrada padrão.
- **--test-sigs=DATABASE TARGET_FILE**: Testa assinaturas do banco de dados em relação ao arquivo de destino.
- **--vba=FILE**: Extrai código VBA/Word6 de um arquivo.
- **--vba-hex=FILE**: Extrai código VBA/Word6 com valores hexadecimais.
- **--diff=OLD NEW ou -d OLD NEW**: Cria um arquivo diff para bancos de dados antigos e novos.
- **--compare=OLD NEW ou -c OLD NEW**: Mostra a diferença entre os arquivos antigo e novo no formato cdiff.
- **--run-cdiff=FILE ou -r FILE**: Executa um script de atualização no diretório atual.
- **--verify-cdiff=DIFF CVD/CLD**: Verifica um arquivo diff em relação a um CVD/CLD.

- **--tempdir=DIRECTORY**: Define um diretório específico para arquivos temporários criados durante o processamento, útil em ambientes com restrições de permissão.
- **--leave-temps[=yes/no(*)]**: Impede a remoção automática de arquivos temporários após o uso, facilitando depuração. O padrão é "no", mas pode ser ativado com --leave-temps=yes.

4. Veja a lista de assinatura de malware que o ClamAV usa (não espere que este comando acabe para continuar com os próximos passos):

```
┌──(root㉿kali)-[~]
└─# sigtool --list-sigs
BC.Legacy.Exploit.CVE_2010_3333-5.{Exploit-CVE_2010_3333}
BC.Legacy.Exploit.CVE_2011_0090-1.{Exploit-CVE_2011_0090}
BC.Win.Packer.script2exe-6754169-0.{}
BC.Legacy.Exploit.CVE_2011_0086-1.{Exploit-CVE_2011_0086}
BC.Legacy.Exploit.CVE_2011_4373-2
BC.Img.Exploit.CVE_2017_12099-6336630-0.{}
...
```

5. Conheça uma das vulnerabilidades mostradas na lista de assinatura encurtada no passo anterior abrindo o Mozilla Firefox (clique em “Aplicativos” → “Navegador web”) e visitando os sites em abas diferentes:

```
https://nvd.nist.gov/vuln/detail/CVE-2017-12099
```

```
https://www.cvedetails.com/cve/CVE-2017-12099/
```

6. Veja que o ataque efetuado por esse Exploit é o Integer Overflow (**NÃO SE ESQUEÇA DE PRINTAR ESTE PASSO!**).
7. Pesquise outros Exploits de acordo com o CVE respectivo.
8. Feche o Firefox ESR e o Terminal. Parabéns! Agora você sabe como funcionam os Antimalwares baseados em Assinatura!

Atividade 6.5 – Automação de Tarefas com PowerShell no Windows Server

Nesta atividade, vamos explorar scripts no PowerShell para automatizar tarefas no Windows Server 2022.

Vamos começar inicializando o Windows Server 2022 (cliente) via RDP ao IP: 192.168.98.30 (certifique-se que o nome de usuário é “WINSERVER\Administrator”), com usuário “administrator” e senha “RnpEsr123@”.

1. Clique no campo “Type here to search” na barra de tarefas e escreva:

```
Windows PowerShell
```

2. Clique no botão “Windows PowerShell”, o Windows PowerShell será aberto.
3. Escreva o seguinte script (copie e cole todo o texto de uma vez só) para coletar informações de sistema e hardware (aperte “Enter” após colar o texto):

```
# Coleta informações do sistema e hardware
$systemInfo = Get-CimInstance -ClassName Win32_ComputerSystem
$osInfo = Get-CimInstance -ClassName Win32_OperatingSystem
$cpuInfo = Get-CimInstance -ClassName Win32_Processor
$memoryInfo = Get-CimInstance -ClassName Win32_PhysicalMemory

# Exibe informações no console
Write-Host "Informações do Sistema:"
Write-Host "Nome do Computador: $($systemInfo.Name)"
Write-Host "Sistema Operacional: $($osInfo.Caption)"
Write-Host "Arquitetura do Sistema: $($osInfo.OSArchitecture)"
Write-Host "Processador: $($cpuInfo.Name)"
Write-Host "Memória Total: $($memoryInfo.Capacity / 1GB) GB"
```

4. A saída do script mostrado acima será:

```
PS C:\Users\Administrator> Write-Host "Informações do Sistema:"
Informações do Sistema:
PS C:\Users\Administrator> Write-Host "Nome do Computador:
$($systemInfo.Name)"
Nome do Computador: WINCLIENT
PS C:\Users\Administrator> Write-Host "Sistema Operacional:
$($osInfo.Caption)"
Sistema Operacional: Microsoft Windows Server 2022 Datacenter
PS C:\Users\Administrator> Write-Host "Arquitetura do Sistema:
$($osInfo.OSArchitecture)"
Arquitetura do Sistema: 64-bit
PS C:\Users\Administrator> Write-Host "Processador: $($cpuInfo.Name)"
Processador: AMD EPYC 7571
PS C:\Users\Administrator> Write-Host "Memória Total:
$($memoryInfo.Capacity / 1GB) GB"
```

O commando mostrado acima mostra os seguintes dados:

- **$systemInfo = Get-CimInstance -ClassName Win32_ComputerSystem**: Nesta linha, o script usa o cmdlet Get-CimInstance para coletar informações sobre o sistema, como o nome do computador. O resultado é armazenado na variável $systemInfo.
- **$osInfo = Get-CimInstance -ClassName Win32_OperatingSystem**: Aqui, o script usa Get-CimInstance para coletar informações sobre o sistema operacional, como a versão do sistema operacional. O resultado é armazenado na variável $osInfo.
- **$cpuInfo = Get-CimInstance -ClassName Win32_Processor**: Nesta linha, o script usa Get-CimInstance para coletar informações sobre a CPU do sistema, como o nome do processador. O resultado é armazenado na variável $cpuInfo.
- **$memoryInfo = Get-CimInstance -ClassName Win32_PhysicalMemory**: Aqui, o script usa Get-CimInstance para coletar informações sobre a memória física do sistema. O resultado é armazenado na variável $memoryInfo.
- **Write-Host "Informações do Sistema:"**: Esta linha exibe o título "Informações do Sistema" no console usando o cmdlet Write-Host.
- **Write-Host "Nome do Computador: $($systemInfo.Name)"**: Aqui, o script usa Write-Host para exibir o nome do computador, que foi coletado anteriormente e está armazenado na variável $systemInfo.Name.

- **Write-Host "Nome do Computador: $($systemInfo.Name)"**: Este comando exibe o nome do computador. A variável $systemInfo.Name contém o nome do computador, que aqui é "WINCLIENT".
- **Write-Host "Sistema Operacional: $($osInfo.Caption)"**: Este comando exibe o nome do sistema operacional. A variável $osInfo.Caption contém o nome do sistema operacional, que aqui é "Microsoft Windows Server 2022 Datacenter".
- **Write-Host "Arquitetura do Sistema: $($osInfo.OSArchitecture)"**: Este comando exibe a arquitetura do sistema operacional (32-bit ou 64-bit). A variável $osInfo.OSArchitecture contém essa informação, que aqui é "64-bit".
- **Write-Host "Processador: $($cpuInfo.Name)"**: Este comando exibe o nome do processador. A variável $cpuInfo.Name contém o nome do processador, que aqui é "AMD EPYC 7571".
- **Write-Host "Memória Total: $($memoryInfo.Capacity / 1GB) GB"**: Este comando exibe a quantidade total de memória do sistema em gigabytes. A variável $memoryInfo.Capacity contém a quantidade total de memória em bytes. A divisão por 1GB converte essa quantidade para gigabytes. Aqui, a memória total é de 4 GB.

5. Agora, vamos escrever um script que coleta informações sobre eventos do sistema a partir do registro de eventos do Windows e gera um relatório em um arquivo de texto. Escreva o seguinte script (copie e cole todo o texto de uma vez só e aperte “Enter” após colar o texto):

```
# Coleta informações de eventos de inicialização do sistema
$events = Get-WinEvent -LogName System | Where-Object { $_.Id -eq 6005
-or $_.Id -eq 6006 }

# Caminho para o arquivo de relatório
$reportFile = "C:\Relatorio_Eventos_Inicializacao.txt"

# Gera um relatório e o salva em um arquivo
$events | ForEach-Object {
    $eventTime = $_.TimeCreated
    $eventMessage = $_.Message
    $report = "Data e Hora: $eventTime`nMensagem: $eventMessage`n`n"
    $report | Out-File -Append -FilePath $reportFile
}

Write-Host "Relatório de eventos de inicialização do sistema salvo em
$reportFile"
```

6. A saída do script mostrado acima será:

```
Relatório de eventos de inicialização do sistema salvo em C:
\Relatorio_Eventos_Inicializacao.txt
```

O commando mostrado acima mostra os seguintes dados:

- **$events = Get-WinEvent -LogName System | Where-Object { $*.Id -eq 6005 -or $*.Id -eq 6006 }**: Nesta linha, o script usa o cmdlet Get-WinEvent para coletar informações de eventos do log "System" do registro de eventos do Windows. Em seguida, ele usa o cmdlet Where-Object para filtrar os eventos com IDs 6005 (inicialização do sistema) ou 6006 (desligamento do sistema) e armazena esses eventos na variável $events.
- **$reportFile = "C:\Relatorio_Eventos_Inicializacao.txt"**: Nesta linha, o script define o caminho para o arquivo de relatório. Neste caso, o relatório será salvo no diretório raiz do disco C:\ com o nome "Relatorio_Eventos_Inicializacao.txt".
- **$events | ForEach-Object {**: Esta linha começa um loop ForEach-Object, que irá iterar sobre cada evento armazenado na variável $events.
- **$eventTime = $_.TimeCreated**: Aqui, o script armazena a data e hora do evento atual na variável $eventTime. A propriedade .TimeCreated contém a data e hora em que o evento ocorreu.
- **$eventMessage = $_.Message**: O script armazena a mensagem do evento atual na variável $eventMessage. A propriedade .Message contém o texto da mensagem do evento.
- **$report = "Data e Hora: $eventTimenMensagem: $eventMessagenn"**: Nesta linha, o script cria uma string de relatório que combina a data e hora do evento com a mensagem do evento, separadas por uma quebra de linha (n). Além disso, há uma linha em branco (n`) após cada evento no relatório.
- **$report | Out-File -Append -FilePath $reportFile**: Aqui, o script usa o cmdlet Out-File para escrever a string de relatório no arquivo especificado (em modo de anexação, -Append). Isso garante que os eventos sejam adicionados ao arquivo sem substituir os eventos anteriores, caso o arquivo já exista.
- **Write-Host "Relatório de eventos de inicialização do sistema salvo em $reportFile"**: Por fim, o script exibe uma mensagem no console usando o cmdlet Write-Host, informando que o relatório de eventos de inicialização do sistema foi salvo no arquivo especificado.

7. Finalmente, na barra de tarefas, abra o “File Explorer”, navegue até o disco C e abra o arquivo “Relatorio_Eventos_Inicializacao.txt”.
8. Neste arquivo txt é possível visualizar os logs de quando o seu Windows Server foi inicializado (**NÃO SE ESQUEÇA DE PRINTAR ESTE PASSO!**).
9. Feche o arquivo “Relatorio_Eventos_Inicializacao.txt” e apague esse arquivo (Shift + Del). Feche o “File Explorer” e o “Windows PowerShell”.

Parabéns! Agora você sabe como usar o PowerShell para automatizar algumas tarefas no Windows Server 2022.